

Agent-a-Thon — note from the judges

Submitted by: Navin

Thanks for putting this in. **You are through to the next round.**

What you need to do next

1. **Read the feedback below.**
 2. **Go through [docs/PREP for AGENT-A-THON.md](#)** in the repo, if you have not already.
 3. **Watch the Info Session — *Deconstructing 2nd-Gen Agentic Architecture*, by Raj Simhan** — if you have not already attended. For a quicker read, the illustration at [docs/One Dinner Four Kitchens.pdf](#) in the repo covers the same ground.
 4. **Submit your AgentSpecs by Tuesday, 15 September, 6:00 pm IST.** Use the template at [docs/SPEC-TEMPLATE.md](#) in the repo. Submissions that do not follow the template cannot be assessed.
 5. **Refer to the [demo/](#) folder in the repo for a worked sample** of a completed spec.
-

What we understood you to be building

A classroom quiz system where five agents track what was taught, what students got wrong, and why—then resurface unresolved gaps in later quizzes until they're fixed. Teachers approve all recommendations; memory persists across sessions in a three-level model (Taught → Assessed → Mastery).

While you have proposed a mobile app, OCR, audio transcription, and five agents tracking taught-assessed-mastery state, please think about what you would be able to build and test with live users in the two days.

Below are a couple of suggestions drawn from your broader idea. Please treat these as examples — decide on the agent or agents you want to build, and create your AgentSpecs in the template provided.

Suggested Agent — Sample 1: Gap Resurfacers

A CLI or web tool that takes quiz results, diagnoses misconceptions, and generates a follow-up quiz targeting unresolved gaps—with teacher approval before anything goes to students.

Reads	A CSV or JSON of quiz results (student, question, answer, correct/incorrect) and a simple topic list the teacher provides.
Remembers between runs	A per-student gap record: which concepts were missed, when, and whether a later quiz resolved them. Stored in a JSON file or Firestore collection.
Sends work backwards	The Diagnosis step proposes 'this wrong answer suggests misconception X.' A second step reviews the diagnosis against the student's history—if the same gap was already marked resolved, it flags the diagnosis as possibly wrong and sends it back for re-evaluation.
Asks a person	Before a follow-up quiz is generated, the teacher sees the proposed gaps and recommended questions. They approve, edit, or dismiss. If they ignore it, the system waits—nothing goes to students without sign-off.

Day one — Ingest a quiz result file, run diagnosis to group wrong answers by misconception, store gaps per student, and display them. No generation yet—just the memory layer working.

Day two — Add the quiz generator that reads the gap record and proposes questions. Add the teacher approval screen. Run it with a real class and watch what happens.

Set aside for the Agent-a-Thon — No mobile app. No live in-class quiz hosting. No audio transcription. No OCR. No student-facing interface beyond receiving the follow-up quiz link.

The core insight—gaps should resurface until fixed—can be proven with one agent, one memory layer, and a teacher in the loop, without building Kahoot from scratch.

Suggested Agent — Sample 2: Misconception Grouper

A tool that takes a set of wrong answers from a single quiz and clusters them by underlying misconception, so the teacher sees 'these 12 students all misunderstand the same thing' instead of 12 separate errors.

Reads	The quiz questions, correct answers, and all student responses (as a spreadsheet or form export).
Remembers between runs	A log of misconceptions identified per quiz, so over time the teacher can see which misunderstandings keep appearing across different quizzes.
Sends work backwards	After the LLM proposes misconception clusters, a review step checks whether any two clusters are actually the same idea phrased differently. If yes, it merges them and re-runs the grouping.
Asks a person	The teacher sees the proposed clusters and can rename, merge, or split them before they're saved. If they do nothing, the clusters are saved as-is after a timeout or explicit 'looks fine' click.

Day one — Parse a quiz export, send wrong answers to the LLM with a prompt that asks for misconception grouping, display the clusters, and let the teacher edit them.

Day two — Add persistence: save clusters to a file or database, show history across quizzes, and surface 'this misconception appeared in 3 of your last 5 quizzes' as a simple report.

Set aside for the Agent-a-Thon — No quiz generation. No student-facing anything. No gap tracking per student—this version groups errors, not learners. No mobile app.

Misconception grouping is the novel part of the idea and can be validated with one LLM call, one review loop, and a teacher who has quiz data on their laptop.